Zeluvi – Estratégia de Escalabilidade

Meta de carga e premissas

O sistema deve sustentar 1 milhão de escritas para cada 2 milhões de leituras, uma proporção de 1 para 2. É muita escrita para um sistema que costuma ser dominado por leitura (a proporção típica fica perto de 1 para 10), então cache e réplicas de leitura ajudam menos e a estratégia de escrita é o que decide o desenho.

Premissas, a confirmar com a equipe:

- **Período:** o enunciado não informa. Usamos "por dia" como cenário base e "por hora" (24 vezes maior) como cenário de estresse.
- **Pico:** 10 vezes a média, porque o uso se concentra em manhã e noite e em eventos como aviso urgente ou assembleia.
- **Escrita:** qualquer inserção ou atualização, incluindo ocorrências, reservas, confirmações de leitura de aviso, status de encomenda, eventos de automação e registros de auditoria.
- **Tamanho:** cerca de 1 KB por registro escrito.

Estimativa de carga

No cenário base (por dia) o sistema recebe cerca de 12 escritas e 23 leituras por segundo na média; no cenário de estresse (por hora) o pico chega a 2.778 escritas e 5.556 leituras por segundo. A arquitetura é projetada para o pico do cenário de estresse, o que deixa o cenário base com folga de cerca de 24 vezes.

Cenário	Escritas/s (média)	Leituras/s (média)	Escritas/s (pico, 10×)	Leituras/s (pico, 10×)
Base: 1 mi de escritas e 2 mi de leituras por dia	11,6	23,1	116	231
Estresse: 1 mi de escritas e 2 mi de leituras por hora	277,8	555,6	2.778	5.556

Crescimento dos dados. Um milhão de escritas por dia a cerca de 1 KB somam aproximadamente 1 GB por dia, ou 365 GB por ano antes dos índices, que costumam dobrar o espaço. É um limite superior, porque parte das escritas atualiza registros que já existem.

Camada de aplicação

A aplicação escala adicionando réplicas, porque é stateless: um balanceador distribui as requisições e o autoescalonamento liga ou desliga réplicas conforme o uso.

- **Réplicas:** no mínimo duas, em zonas de disponibilidade diferentes. O máximo é definido pelo teste de carga.
- **Autoescalonamento:** por uso de CPU (alvo de 60%) e por requisições por réplica. Réplicas novas ficam prontas em segundos, pois não carregam estado.
- **Saúde:** o balanceador testa cada réplica e tira da rota a que falhar.
- **Proteção:** limite de requisições por usuário e por IP, contado no Redis, evita que um único cliente consuma a capacidade de todos.
- **Dimensionamento:** a capacidade de uma réplica será medida no teste de carga. O número de réplicas de partida é o pico do cenário de estresse dividido por essa capacidade, mais 30% de folga.
- **Workers:** processos separados que consomem as filas e escalam pelo tamanho da fila, sem competir com a API.

Estratégia de leitura

Com duas leituras para cada escrita, a leitura é resolvida em três degraus, e cada um tira carga do banco principal.

1. **Cache em Redis (cache-aside).** Dados muito lidos e que mudam pouco, como avisos recentes, regras de reserva e indicadores do painel, ficam em cache com validade curta. O cache é limpo por evento quando o registro muda.
2. **Réplicas de leitura do PostgreSQL.** Consultas que toleram alguns segundos de atraso (histórico, relatórios, painel executivo) vão para as réplicas. Quando o usuário acabou de escrever algo e vai lê-lo (ver a ocorrência recém-criada), a leitura vai ao banco principal para não mostrar dado velho.
3. **CDN para arquivos.** Fotos, anexos e arquivos estáticos do web saem do S3 pelo CloudFront e nunca passam pela API.

Paginação por cursor e respostas enxutas (só os campos da tela) reduzem o volume de dados em qualquer degrau.

Cuidado com a proporção 1 para 2. Como há muita escrita, o cache é invalidado com frequência. Por isso ele só entra onde o dado muda pouco; o status de uma ocorrência em andamento, por exemplo, não é cacheado.

Estratégia de escrita

A escrita é o ponto difícil, porque só o banco primário a aceita. A regra é fazer menos trabalho por escrita e tirar da requisição tudo o que não precisa ser imediato.

1. **Conexões sob controle.** Um pool de conexões (RDS Proxy) impede que muitas réplicas esgotem as conexões do banco.
2. **Separar o imediato do adiável.** A API grava o essencial numa transação curta (por exemplo, a ocorrência) e responde. Notificações, auditoria detalhada, automações e métricas viram eventos numa fila e são processados pelos workers.
3. **Escrita em lote.** Os workers agrupam gravações parecidas, como confirmações de leitura de um mesmo aviso e eventos de auditoria, em poucos comandos no lugar de milhares.
4. **Repetição segura.** Chaves de idempotência e restrições de unicidade permitem reprocessar uma mensagem da fila sem duplicar registros.
5. **Particionamento por mês.** As tabelas que mais crescem (confirmações de leitura, auditoria e histórico) são particionadas por mês. Partições antigas vão para armazenamento mais barato e os índices ativos ficam pequenos.
6. **Escalar o primário em etapas.** Primeiro uma instância maior; depois, mover as tabelas de maior volume para um banco próprio; só por último dividir os condomínios entre bancos (sharding), e apenas se o teste de carga mostrar que é necessário.
7. **Fila sob pressão.** Se a fila atrasar além do limite definido, o autoescalonamento sobe mais workers e um alerta avisa a equipe.

Gargalos previstos e mitigações

O primeiro limite a aparecer deve ser o banco primário, porque ele concentra todas as escritas. Os demais pontos de atenção estão na tabela, cada um com o sinal que indica o problema e a resposta planejada.

Gargalo	Sinal de alerta	Mitigação
Banco primário (CPU e disco de escrita)	Latência de escrita acima da meta	Escrita em lote, partições, instância maior e, depois, tabelas de alto volume em banco próprio
Conexões ao banco	Erros de limite de conexões	Pool de conexões e limite de conexões por réplica
Cache pouco eficaz	Taxa de acerto baixa	Cachear só dados estáveis, com chaves por condomínio

Gargalo	Sinal de alerta	Mitigação
Pico de notificações (aviso urgente a todos os moradores)	Fila de notificações crescendo	Workers escalam pelo tamanho da fila; envio em lotes e em ondas
Réplicas de leitura atrasadas	Atraso de replicação crescente	Ler do primário quando o dado é recente; adicionar réplicas
Chamadas de IA lentas ou caras	Tempo de resposta e custo por condomínio	Sempre assíncronas, com limite de uso por condomínio
Tabelas muito grandes (auditoria, leituras de aviso)	Consultas lentas e índices grandes	Partição mensal e arquivamento das partições antigas

Testes de carga e plano de crescimento

A meta só vale se for medida. Antes da defesa e a cada release relevante, cinco testes (com a ferramenta k6) confirmam que o sistema sustenta o pico do cenário de estresse, sempre mantendo a proporção de 1 escrita para 2 leituras.

Teste	O que faz	Critério de aprovação
Carga normal	278 escritas/s e 556 leituras/s por 1 hora	p95 abaixo de 300 ms na leitura e de 500 ms na escrita; menos de 0,1% de erros
Pico	2.778 escritas/s e 5.556 leituras/s por 15 minutos	Mesmas metas da carga normal
Além do pico	Sobe até o dobro do pico	Revela o primeiro gargalo e o limite do sistema
Resistência	Carga média por 8 horas	Memória e latência estáveis ao longo do tempo
Falha	Derruba uma réplica da aplicação e o banco primário durante a carga	Recuperação sem perda de dados e dentro do tempo de retorno de 1 hora

Gatilhos de crescimento (valores iniciais, a ajustar depois dos testes)

- CPU do banco primário acima de 60% por 15 minutos: subir a instância ou mover tabelas de alto volume para um banco próprio.
- Atraso de replicação acima de alguns segundos: adicionar réplicas de leitura.

- Fila de mensagens crescendo por vários minutos seguidos: aumentar o limite de workers.
- Taxa de acerto do cache baixa: rever o que é cacheado antes de aumentar o Redis.